

char word[10];	char *cptr;	cptr = word;
cptr	word	&word[0]
cptr + n	word + n	&word[n]
*cptr	*word	word[0]
*(cptr + n)	*(word + n)	word[n]

Comp: CPU executes instructions using ALU and registers; Memory stores both instructions and data; System bus & memory Bus interconnect CPU, Memory, and I/O; I/O bus connects I/O bridge to peripherals; I/O peripherals keyboard, mouse, etc. Moore's Law: number of transistors on a chip doubles every 18 months, processor speed doubles every 18 months, memory capacity doubles every two years, disk capacity doubles every year. Processor-Memory Speed Gap: processor-memory performance gap grows 50% per year → memory hierarchy (registers, caches). Power Wall: Energy limits stopped clock speed growth, forcing a shift to multicore CPUs and limiting single-threaded speedups.

Binary: Have only 0 and 1. Signed magnitude: leftmost bit is the sign bit; the rest represent magnitude. 1's Complement: Flip all bits to negate. First bit act as a sign bit. Add 1 if overflowed. Have distinct -0 & 0 ($\max = 2^{n-1} - 1$, $\min = -2^{n-1} + 1$). 2's Complement: Flip all bits and add 1 to negate ($\max = 2^{n-1} - 1$, $\min = -2^{n-1}$). Have no -0. Cannot negate extra negative value. Overflow: Unsigned overflow is when correct ans requires too many bits (i.e., $a + b < a$ or $a + b < b$). Signed overflow is when we carry into sign bit (e.g., $a + b = c$, $a > 0$ and $b > 0$ but $c < 0$). Hexadecimal: Digits are 0-9 and A-F (represent 10-15). Octal: Digits are 0-7.

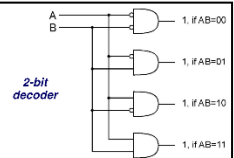
IEEE Floating points: Have 3 parts (sign bit, exponent, significand), implicit int = 1. Exponent is signed number, represent as biased integers (E bits for exponent → bias = $2^{E-1} - 1$), $e = \text{expo} - \text{bias}$. Zero values: all zero bits for exponent and significand → large gap between smallest positive and zero. Subnormal values: have all-0 exponent, implicit int = 0, $e = 1 - \text{bias}$. Infinite values: have all-1 exponents, use for special error values. If significand is all-0: value = $\pm\infty$. Else: value = NaN. Special value: $x / 0 = \infty$ ($x > 0$) and $0 / 0 = \infty - \infty = \sqrt{-1} = \text{NaN}$, etc. Any arithmetic / comparisons with NaN returns NaN / False. Ranges: Single w/ 8 expo bits, 23 significand bits, normal expo -126 → 127, ULP range from 2^{-149} → 2^{104} . Double w/ 11 expo bits, 52 significand bits, normal expo -1022 → 1023, ULP range from 2^{-1074} → 2^{971} . ULP (Unit in Last Place) is the gap between successive values of an expo, value of the least significant bit in significand, 2^{-e} , for expo e and S-bit significand.

Assembly: Minimal data types (Integers data of 1, 2 or 4 bytes with data values and addresses; Floating point data of 4, 8 or 10 bytes; No arrays, structures, etc.), No type checking (Interpretation of data formats depends on instruction, No protection against misinterpretation of data), with 3 types of primitive operations (perform arithmetic func on register or memory data, transfer data between memory and register, transfer control with unconditional jumps to / from procedures and conditional branches). X86: variable length instruction: 1-15 bytes, address memory directly, use Little-Endian format (least significant byte in the lowest address). Instruction: opcode operand. Opcode is short mnemonic for instruction purpose (movb, addl, etc.). Operands are immediate register or memory. Register names: RAX (EAX₄, AX₂, AH₁, AL₁), RBX, RCX, RDX, RSP (ESP₄, SP₂), RBP, RSI, RDI, RIP, R8(R8D₄, R8W₂, R8B₁) R9, ..., R15. IA32 push: pushl src → fetch operand at src → %esp → 4 → write operand at (%esp). IA32 pop: popl dest → read operand at (%esp) → %esp += 4 → write to dest. Procedure control: call label (push address on stack, jump to label) and ret (pop address from stack, jump to address). Operations: instruction src, dest → dest = foo(dest, src). E.g., addl a, b ($b = b + a$) or subl a, b ($b = b - a$) or imull a, b ($b = b * a$). Similarly, there're sall (<<), sarl (>>), xori (^), andl (&), orl (|). Additionally, incl a ($a += 1$), decl a ($a -= 1$), negl a ($a = -a$), notl a ($a = \sim a$). For expressions, we can use leal src, dest (e.g. leal 7(%edx) %eax → %eax = %edx + 7). For comparison we can use cmpl a, b (compute $b - a$);

Digital logic: AND (A & B), OR (A || B), NOT (!A), NAND (!A & B), NOR (!A || B), XOR (A & !B) || (!A & B). Minterms: A product term which all var appear once (complemented or uncomplemented). Decoder: n inputs, 2^n outputs (exactly 1 outputs for each is 1 for each input patterns). "Standard" decoder: i^{th} output is 1 and others are 0. Multiplexer (MUX): n-bit selector, 2^n input, 1 output. Sequential circuits: has outputs that depend on current and prior values. State: all information about a circuit to explain its future behavior. Latches & flipflops: state elements stored on 1 bit of state. Synchronous sequential circuits: combinational logic + bank of flip flops. Latch: controlled by level, has enable/control signal (Active signal → transparent latch immediate output. Inactive signal → opaque latch, hold previous values). S/R Latch: $S = 1 \rightarrow Q = 1$, $R = 1 \rightarrow Q = 0$, invalid when $S = R$. D Latch: C is the signal, D is the values, Q is the output (see Latch's properties). Flip flop: latches on a clock, synchronous. D flip flop: has 2 inputs (D and clock). Clock = 0 → primary enabled (N1 obtains primary's input), secondary disabled (old output is still output). Clock = 1 → primary disabled (N1 is old value), secondary enabled (N1 copied to output). Latch vs flip flop: latch outputs change at any time, flip flops only change during clock transitions. Finite state machines: State registers + combinational logic (stores next state and loads next state @ clock edge + computes the next state and outputs).

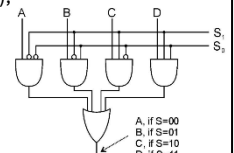
Caches: bridge large speed gap between CPU and main memory. CPU speed > DRAM speed → need SRAM caches to improve performance. Memory hierarchy: layered storage, organized by speed, costs, size: L0 (registers) → L1 caches → L2 caches → main memory → disks. Cache hit: requested data is found in cache. Cache miss: data is not in the cache and must be fetched from lower level with extra delay. Cold (compulsoty) miss: occurs the 1st time a memory block is access. Capacity miss: when program's working set is larger than the cache → blocks are evicted despite would be useful. Conflict miss: multiple memory blocks map to the same cache set → repeated evictions despite available spaces elsewhere in the cache. Locality: temporal locality (recently used data is likely to be used again) & spatial locality (data near recently used data is likely to be accessed next). Division and arrangement: main memory is divided into fixed-size blocks (unit of data transferred between memory and cache), caches are divided into sets, each contains 1+ lines (a memory blocks maps to exactly 1 set but can occupy any line within that set) Block size: $B = 2^b$ bytes, larger B improve spatial locality but increase miss penalty. Sets: $S = 2^s$ sets in a cache. Associativity: E lines per set (direct mapped → $E = 1$, fully associativity → $E = n$ cache lines), higher E reduces conflict miss but increase hit time and complex. Cache size: $C = B * E * S$ bytes, larger C reduce capacity miss but are costly and slower. Address: has a given n bits, where $n = t + s + b$, consist of tag (t bits) + set index (s bits) + block offset (b bits)

Note: unless specified below, arguments & return values are int's
char buffer[], ch, *name, *ptr, *s_mode;
long offset;
struct stat *stat_buf;
FILE *stream;



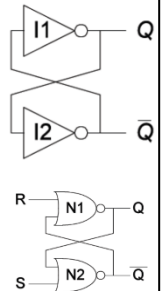
open (name,mode);
read(filesdes,buffer,count);
lseek(filesdes,offset,from);
creat(name, mode);
FILE * fopen (name,s_mode);
FILE * freopen(name,s_mode,stream);
fread(ptr,item_size,count,stream);
fwrite(ptr,item_size,count,stream);
getc(stream);
putc(ch,stream);
seek(stream,offset,from);

0: read, 1: write, 2: both
write(filesdes,buffer,count);
0: begin, 1: current, 2: end
close(filesdes);
"r": read, "w": write, "a": append
FILE * fdopen(filesdes,s_mode);
char * gets(buffer);
puts(buffer);
getchar();
putchar(ch);
fclose(stream);



printf(format,exp1,exp2, ...)
fprintf(stream,format,exp1,exp2, ...)
sprintf(buffer,format,exp1,exp2, ...)
scanf(format,addr1,addr2, ...)
fscanf(stream,format,addr1,addr2, ...)
sscanf(buffer,format,addr1,addr2, ...)

to standard output
to specified output
to string buffer
from standard input
from specified input
from string buffer



Note: Destination addresses are required with scanf, fscanf & sscanf
Format string consists of text to be printed or matched containing format specifiers that has the form: % [-] [*] [W] [.M] [I] <conversion character>

Where:

%	W	width in characters
-	M	precision (printf)
*	I	specifies long int or double

Conversion character:

d	c	single char
u	s	null terminated str
x	f	fixed point notation for float or double
h	e	notation for float or double (printf)
o	g	%e or %f, whichever shorter (printf)